

FATEC Ourinhos

Análise e Desenvolvimento de Sistemas — 3.º Semestre

SISTEMAS OPERACIONAIS

Resumo Completo para Prova

Material de Estudo com Simulado

1. Fundamentos de Sistemas Operacionais

Um Sistema Operacional (SO) atua em duas dimensoes fundamentais: como maquina estendida, abstraindo o hardware e oferecendo ao programador uma interface de alto nivel; e como gerenciador de recursos, arbitrando o acesso concorrente de processos a CPU, memoria e dispositivos. Sem o SO, cada programa precisaria controlar diretamente o hardware — caotico, inseguro e incompativel entre maquinas.

Modos de Operacao: Kernel vs. Usuario

Processadores modernos operam em (no minimo) dois modos de privilegio. No modo kernel (supervisor), o SO executa com acesso irrestrito ao hardware. No modo usuario, os processos comuns executam com acesso limitado. A transicao de usuario para kernel ocorre exclusivamente por interrupcao, execucao ou chamada de sistema (syscall) — nunca diretamente.

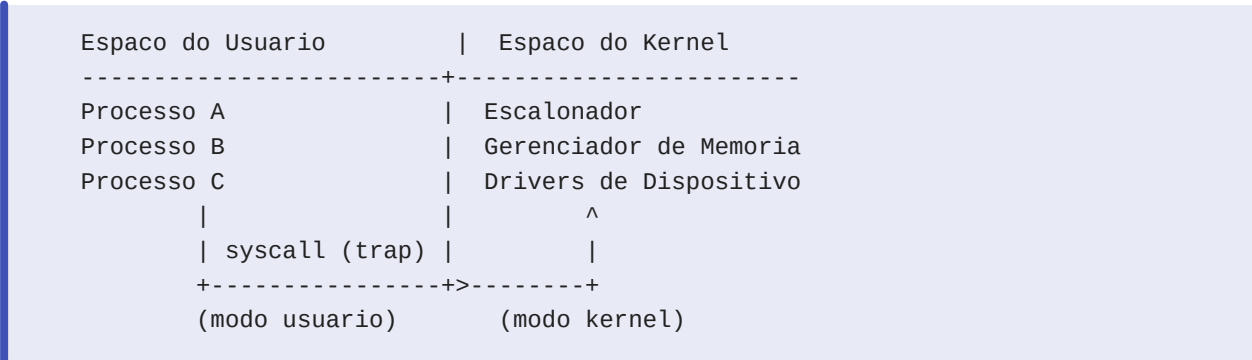


Figura 1 — Transicao entre modos via syscall (trap)

Chamadas de Sistema (Syscalls)

Syscalls sao a interface controlada entre o espaco do usuario e o kernel. Quando um processo precisa de um servico privilegiado (abrir arquivo, alocar memoria, criar processo), ele executa uma instrucao de trap que transfere o controle para o kernel, que valida e executa a operacao. No Linux, exemplos comuns incluem: read(), write(), fork(), exec(), mmap(), exit(). A tabela de syscalls mapeia cada numero de chamada para o handler correspondente no kernel.

Tipos e Arquiteturas de SO

Arquitetura	Caracteristica	Exemplo
Monolitico	Kernel unico; todos os servicos no mesmo espaco	Linux, Unix classico
Microkernel	Kernel minimo; servicos em processos de usuario	MINIX, QNX, Mach

Arquitetura	Característica	Exemplo
Hibrido	Nucleo microkernel com modulos no kernel por desempenho	Windows NT, macOS XNU
Exokernel	Kernel expoe hardware diretamente; libOS gerencia abstracoes	MIT Exokernel

Dica: Na prova: monolitico = mais rapido (sem overhead de IPC), microkernel = mais seguro e modular. Linux usa monolitico com modulos carregaveis (loadable kernel modules).

2. Virtualizacao e Emulacao

Virtualizacao permite que multiplos sistemas operacionais convidados (guests) executem sobre um unico hardware fisico, cada um acreditando ter acesso exclusivo ao hardware. O componente que faz isso se chama Hipervisor (ou VMM — Virtual Machine Monitor). Existem dois tipos fundamentais.

Hipervisor Tipo 1 (Bare-Metal)

Executa diretamente sobre o hardware, sem SO hospedeiro. Tem acesso privilegiado completo ao hardware e menor overhead. Usado em producao e data centers. Exemplos: VMware ESXi, Microsoft Hyper-V, Xen.

Hipervisor Tipo 2 (Hosted)

Executa como processo sobre um SO hospedeiro convencional. Mais facil de instalar e usar, mas com overhead adicional por passar pelo SO hospedeiro. Exemplos: VirtualBox, VMware Workstation, QEMU.

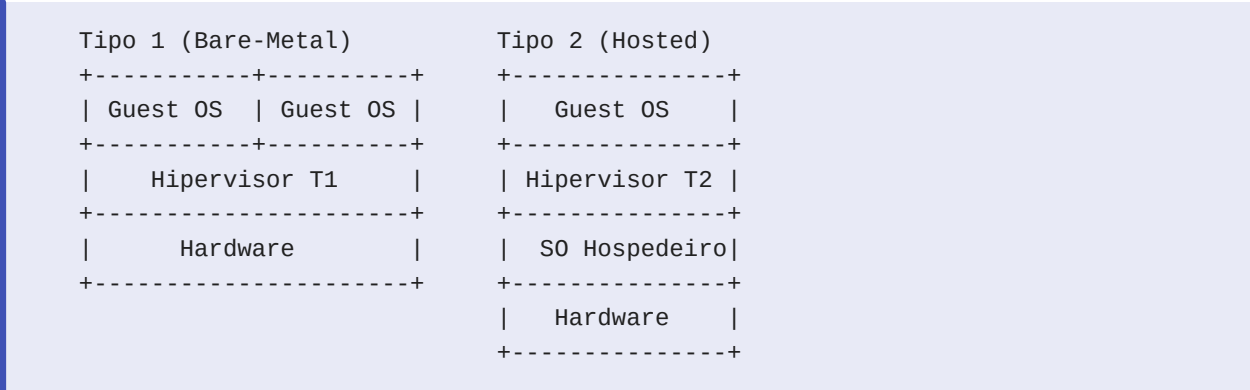


Figura 2 — Hipervisor Tipo 1 vs. Tipo 2

Containers vs. Maquinas Virtuais

Aspecto	Container (Docker)	VM Completa
Isolamento	Namespace do kernel (parcial)	Hardware virtualizado (total)
Overhead	Quase zero — compartilha kernel	Alto — SO completo por VM
Boot	Milissegundos	Dezenas de segundos
Portabilidade	Imagem de app	Imagem de SO completo
Seguranca	Menor isolamento	Maior isolamento

Dica: Containers não são VMs! Um container compartilha o kernel do host via namespaces e cgroups. VMs emulam hardware completo e rodam SO próprio. Use VM quando precisar de isolamento total ou SO diferente; use container para deploy rápido de aplicações.

3. Processos e Escalonamento

Um processo é um programa em execução — a unidade básica de atividade gerenciada pelo SO. É composto por quatro elementos essenciais: (1) conjunto de instruções (o programa), (2) espaço de endereçamento (região de memória), (3) contexto de hardware (registradores, PC, SP) e (4) contexto de software (descritores de arquivo abertos, variáveis de ambiente, UID). O SO mantém um PCB (Process Control Block) para cada processo com todas essas informações.

Estados de um Processo

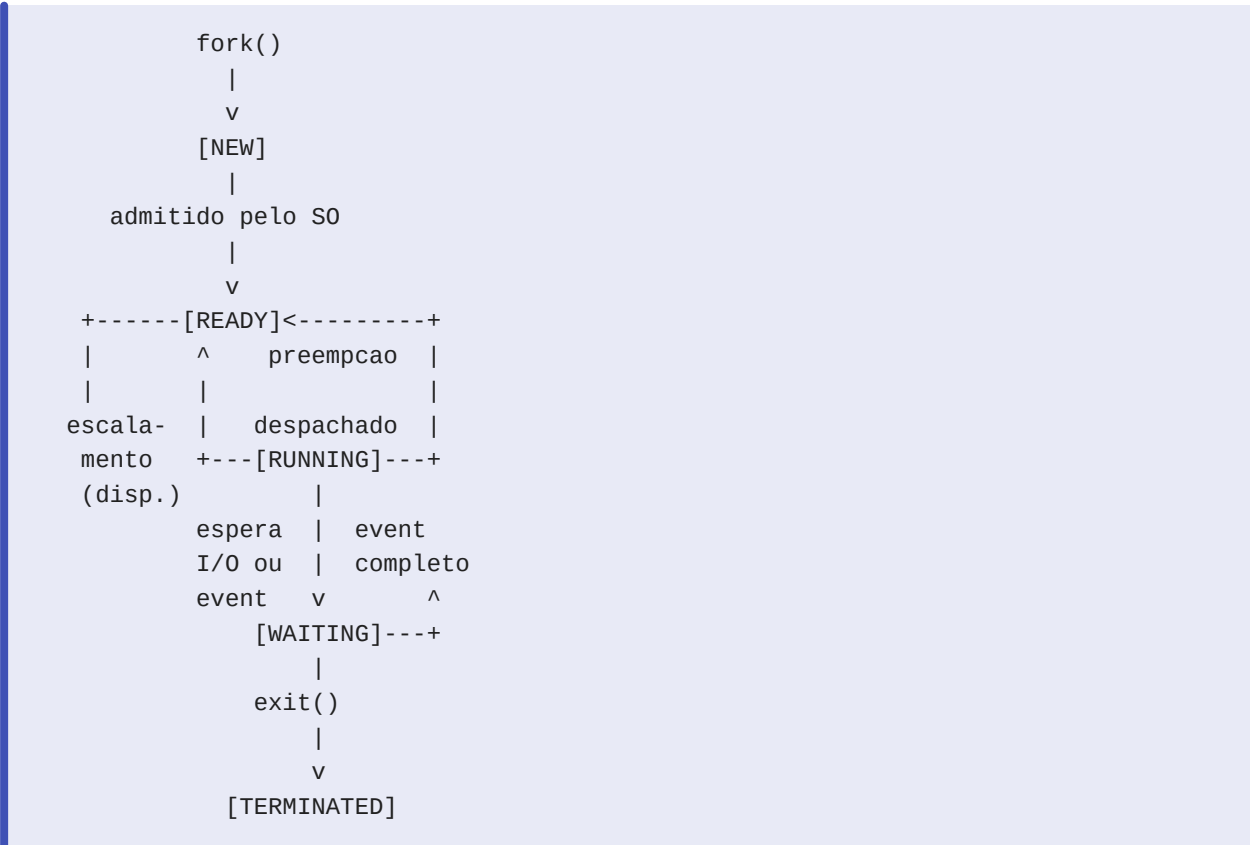


Figura 3 — Diagrama de estados de um processo

Algoritmos de Escalonamento

Algoritmo	Preemptivo?	Caracteristica	Problema
FCFS	Nao	Primeiro a chegar, primeiro a ser servido	Convoy effect: processos curtos bloqueados por longo
SJF	Nao/Sim	Menor job primeiro — minima espera media	Requer conhecer tempo de burst (impraticavel em geral)
Round-Robin	Sim	Cada processo recebe um quantum de CPU	Quantum pequeno = overhead alto; grande = vira FCFS
Prioridade	Sim/Nao	CPU vai ao processo de maior prioridade	Starvation: processos de baixa prioridade nunca executam
Multilevel Feedback	Sim	Filas de prioridade dinamicas adaptativas	Mais complexo de implementar

Round-Robin em Detalhe

Round-Robin é o algoritmo mais usado em sistemas interativos (Linux usa variante chamada CFS). Cada processo recebe um quantum (fatia de tempo — tipicamente 10-100ms). Ao esgotar o quantum, o processo é preemptado e vai para o final da fila READY. Garante fairness: nenhum processo monopoliza a CPU. Trade-off: quantum muito pequeno gera overhead de troca de contexto; quantum muito grande degenera em FCFS e aumenta o tempo de resposta médio.

Starvation (Inanicao)

Ocorre quando um processo nunca é executado porque processos de maior prioridade continuam chegando. Solução clássica: aging (envelhecimento) — a prioridade de um processo aumenta gradativamente com o tempo que ele aguarda na fila, garantindo execução eventual.

4. Threads e Modelos de Concorrencia

Thread é a unidade básica de execução dentro de um processo. Enquanto processos têm espaços de endereçamento independentes, threads dentro do mesmo processo compartilham: espaço de endereçamento, descritores de arquivo, variáveis globais e heap. Cada thread tem seu próprio: contador de programa (PC), pilha de chamadas e registradores. Isso torna a criação de thread muito mais barata que a de processo (fork).

Processo vs. Thread

Aspecto	Processo	Thread
Espaco de enderecamento	Independente (isolado)	Compartilhado no processo
Heap e variaveis globais	Privados	Compartilhados
Pilha	Privada	Privada por thread
Custo de criacao	Alto (fork + copy-on-write)	Baixo
Custo de troca de contexto	Alto (troca TLB/MMU)	Baixo (mesma MMU)
Comunicacao	IPC (pipe, socket, shmem)	Direto via memoria compartilhada
Falha isolada?	Sim (crash nao afeta outros)	Nao (crash mata o processo todo)

Modelos de Thread

Existem tres modelos que descrevem como threads de usuario se mapeiam para threads de kernel:

- N:1 (many-to-one): N threads de usuario mapeadas para 1 thread de kernel. Troca de contexto rapida (feita em userspace), mas uma syscall bloqueante para TODO o processo. Sem paralelismo real (nao usa multiplos nucleos). Ex: Java Green Threads (legado).
- 1:1 (one-to-one): Cada thread de usuario tem uma thread de kernel correspondente. Paralelismo real em multicore. Syscall bloqueante afeta apenas a thread. Mais caro de criar/destruir. Ex: pthreads no Linux, Windows threads.
- M:N (many-to-many): M threads de usuario mapeadas para N threads de kernel ($N \leq M$). Combina vantagens: pool de kernel threads reusado. Implementacao complexa. Ex: Solaris.

Funcao JOIN

`pthread_join(tid, &retval;)` bloqueia a thread chamadora ate que a thread `tid` termine. Serve para dois fins: (1) sincronizacao — garantir que o resultado de uma thread esteja pronto antes de continuar; (2) liberacao de recursos — sem JOIN ou DETACH, os recursos da thread (pilha, bloco de controle) ficam retidos no kernel mesmo apos a thread encerrar (thread zombie). DETACH configura a thread para liberar recursos automaticamente ao terminar, sem precisar de JOIN.

```
#include <pthread.h>
#include <stdio.h>

void* tarefa(void* arg) {
    int id = *(int*)arg;
    printf("Thread %d executando\n", id);
    return (void*)(long)id * 2; // valor de retorno
}

int main(void) {
    pthread_t tid;
    int id = 42;
    pthread_create(&tid, NULL, tarefa, &id);

    void* retval;
    pthread_join(tid, &retval); // bloqueia ate thread terminar
    printf("Retorno: %ld\n", (long)retval);
    return 0;
}
```

Codigo 1 — Criacao e JOIN de thread POSIX

5. Sincronizacao e Exclusao Mutua

Condicao de Corrida (Race Condition)

Ocorre quando dois ou mais processos/threads acessam e manipulam dados compartilhados concorrentemente, e o resultado final depende da ordem especifica de execucao (scheduling). O resultado e imprevisivel e potencialmente incorreto. Exemplo classico: dois processos incrementando um contador compartilhado.

```
// PROBLEMA: race condition no contador
int contador = 0; // variavel compartilhada

// Thread A:                // Thread B:
int tmp = contador;          int tmp = contador; // ambas lem 0
tmp = tmp + 1;                tmp = tmp + 1;           // ambas calculam 1
contador = tmp;               contador = tmp;           // ambas escrevem 1

// Resultado esperado: 2 | Resultado real: 1 (perdeu um incremento!)
```

Codigo 2 — Race condition em contador compartilhado

Regiao Critica e Exclusao Mutua

A regio critica e o trecho de codigo que acessa recursos compartilhados e nao pode ser executado por mais de um processo simultaneamente. Exclusao mutua e a propriedade que garante isso. Quatro condicoes devem ser satisfeitas por qualquer solucao de exclusao mutua: (1) Exclusao mutua: so um processo na regio critica por vez; (2) Progresso: se nenhum processo esta na RC, quem quer entrar pode entrar; (3) Espera limitada: um processo nao espera indefinidamente para entrar; (4) Sem suposicoes sobre velocidade ou numero de CPUs.

Mecanismos de Sincronizacao

Mecanismo	Como funciona	Uso ideal
Busy waiting (spinlock)	Loop ativo verificando flag — desperdicca CPU	Espera muito curta, multicore
Desabilitar interrupcoes	Impede preempcao — nao funciona em multicore	SO monousuario, kernel interno
Variaveis de lock	Flag boolean — sofre race condition na propria flag	Didatico; nao use em producao
Algoritmo de Peterson	Solucao software elegante para 2 processos	Fundamento teorico
Semaforo	Contador atomico com operacoes P (wait) e V (signal)	Producao, n processos, contagem

Mecanismo	Como funciona	Uso ideal
Mutex	Semaforo binario com dono (owner) — so quem travou pode destrair	Proteger recurso unico
Monitor/Condicao	Abstracao de alto nivel com lock implicito	Java synchronized, C# lock

Semaforo vs. Mutex — Diferenca Critica

Semaforo e um contador inteiro nao-negativo com duas operacoes atomicas: P (proberen/wait) decrementa — bloqueia se zero; V (verhogen/signal) incrementa — acorda thread bloqueada se houver. Pode ser usado para contagem de recursos (ex: 5 licencas disponiveis) ou para sincronizacao de ordem entre threads. Mutex (mutual exclusion lock) e especificamente binario (0 ou 1) e tem o conceito de dono: apenas a thread que adquiriu o mutex pode liberado. Isso evita o problema de uma thread liberar o lock de outra (bugs sutis).

```
#include <pthread.h>
#include <semaphore.h>

// --- Mutex ---
pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
int contador = 0;

void* incrementar(void* arg) {
    pthread_mutex_lock(&mtx);    // entra na regioao critica
    contador++;                  // operacao atomicamente protegida
    pthread_mutex_unlock(&mtx);  // sai da regioao critica
    return NULL;
}

// --- Semaforo para sincronizar ordem ---
sem_t sem;
// Thread produtor: sem_post(&sem);
// Thread consumidor: sem_wait(&sem); // bloqueia ate produtor sinalizar
```

Codigo 3 — Mutex e semaforo com pthreads

Dica: Regra pratica: use mutex para proteger um recurso unico (exclusao mutua simples). Use semaforo quando precisar contar recursos (ex: N slots no buffer) ou sincronizar ordem entre threads (ex: consumidor espera produtor gerar dado).

6. Problemas Classicos de Concorrencia

Jantar dos Filósofos

Cinco filósofos sentam em torno de uma mesa circular. Entre cada par de filósofos ha exatamente um garfo. Para comer, um filosofo precisa de dois garfos (o da esquerda e o da direita). O ciclo de cada filosofo: pensar -> pegar garfo esq -> pegar garfo dir -> comer -> largar garfos. O problema modela alocacao de recursos com exclusao mutua e e um benchmark classico para solucoes de deadlock.



Figura 4 — Jantar dos Filósofos: 5 filósofos, 5 garfos

Como surge o Deadlock no Jantar

Se todos os filósofos pegarem o garfo esquerdo simultaneamente, cada um tera um garfo e esperara pelo garfo direito (que esta na mao do vizinho). Resultado: espera circular — nenhum consegue o segundo garfo, todos ficam bloqueados para sempre. As quatro condicoes de Coffman estao presentes: exclusao mutua (garfo so para um), segurar e esperar, nao preempcao, espera circular.

Solucoes para o Jantar dos Filósofos

- Assimetria: filosofo de indice par pega garfo esquerdo primeiro; impar pega direito primeiro. Quebra a espera circular.
- Limite de filósofos: permitir no maximo 4 filósofos a mesa simultaneamente (semaforo inicializado em 4). Garante que pelo menos um consiga ambos os garfos.
- Pegar ambos ou nenhum: verificar disponibilidade dos dois garfos atomicamente antes de pegar qualquer um.
- Solucao de Dijkstra com semaforos: cada filosofo eh um processo, cada garfo eh um semaforo binario. Usar semaforo adicional para coordenacao.

Problema dos Leitores e Escritores

Um banco de dados e acessado por leitores (leitura concorrente e segura) e escritores (escrita exclusiva — nenhum outro pode acessar). Invariante: multiplos leitores podem ler simultaneamente, mas um escritor precisa de acesso exclusivo total. O problema de

starvation dos escritores ocorre na solucao que prioriza leitores: se leitores chegam continuamente, o escritor nunca consegue acesso. Solucao: quando um escritor sinaliza intencao de escrever, novos leitores ficam bloqueados — escritor tem prioridade sobre leitores que chegam apos o pedido de escrita.

Barbeiro Adormecido

Barbearia com 1 barbeiro, 1 cadeira de trabalho e N cadeiras de espera. Se nao ha clientes, o barbeiro dorme. Cliente que chega: (1) se barbeiro dorme, acorda-o; (2) se ha cadeira de espera livre, senta e aguarda; (3) se sala cheia, vai embora. Modela o padrao produtor-consumidor com buffer limitado. A solucao usa semaforos: um para clientes (conta quantos aguardam), um para barbeiro (conta se barbeiro esta disponivel) e um mutex para proteger o contador de clientes.

7. Deadlocks (Impasses)

Um deadlock e uma situacao em que um conjunto de processos ficam bloqueados permanentemente, cada um esperando por um recurso que esta sendo retido por outro processo do conjunto. Nenhum progresso e possivel. O termo 'impasse' em portugues e igualmente usado na literatura brasileira (Tanenbaum).

As 4 Condicoes Necessarias de Coffman (1971)

Para que um deadlock ocorra, TODAS as quatro condicoes abaixo devem estar presentes simultaneamente. Eliminar qualquer uma previne o deadlock:

- 1. Exclusao Mutua: o recurso so pode ser usado por um processo de cada vez (nao e compartilhavel).
- 2. Segurar e Esperar (Hold and Wait): um processo que ja possui pelo menos um recurso fica aguardando por outros recursos que estao em posse de outros processos.
- 3. Nao Preempcao (No Preemption): recursos nao podem ser tomados forcadamente de um processo; so sao liberados voluntariamente por quem os possui.
- 4. Espera Circular (Circular Wait): existe uma cadeia fechada de processos $P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_n \rightarrow P_0$ onde cada P_i espera por um recurso que esta com P_{i+1} .

Dica: Mnemonica para as 4 condicoes: EMSeNE-C — Exclusao Mutua, Segurar e Esperar, Nao preempcao, Espera Circular. Todas necessarias, nenhuma suficiente sozinha.

As 4 Estrategias de Tratamento

Estrategia	Como funciona	Custo	Usado em
Prevencao	Projetar sistema para negar ao menos uma das 4 condicoes sempre	Alto (recursos subutilizados)	Sistemas de seguranca critica
Evasao	Analisar cada pedido e so conceder se estado resultante for seguro (Algoritmo do Banqueiro)	Medio (precisa declarar max)	Sistemas com info de max conhecido
Deteccao e Recuperacao	Permitir deadlocks; detectar ciclos no grafo; abortar/reverter processos	Recuperacao custosa	BDs com transacoes (rollback)
Politica do Avestruz	Ignorar o problema — muito raro para valer a pena tratar	Quase zero	Unix, Windows (na pratica)

Algoritmo do Banqueiro (Banker's Algorithm)

Proposto por Dijkstra, o Algoritmo do Banqueiro implementa a estrategia de evasao. Cada processo declara antecipadamente o numero maximo de recursos que pode precisar. Quando um processo solicita recursos, o algoritmo simula a concessao e verifica se o estado resultante e 'seguro' — ou seja, se existe alguma sequencia de finalizacao em que todos os processos conseguem seus recursos maximos e terminam. Se o estado for inseguro, a requisicao e negada e o processo aguarda. Limitacao pratica: processos raramente conhecem seus maximos de antemao, tornando-o impraticavel em sistemas gerais.

Prevencao — Negando Cada Condicao

- Negar Exclusao Mutua: tornar recursos compartilhaveis quando possivel (ex: leitura de arquivo — multiplos leitores). Nem sempre possivel (impressora nao pode ser compartilhada em uso).
- Negar Segurar e Esperar: exigir que processos solicitem TODOS os recursos necessarios de uma vez (tudo ou nada). Problema: baixa utilizacao, starvation possivel.
- Negar Nao Preempcao: permitir que o SO force a liberacao de recursos. Funciona para recursos cujo estado pode ser salvo/restaurado (CPU, memoria). Nao funciona para impressora.
- Negar Espera Circular: impor ordenacao total dos recursos. Todo processo deve solicitar recursos em ordem crescente. Ex: se $R1 < R2$, so pode pedir R2 depois de pegar R1. Elimina ciclos no grafo.

SIMULADO

Questão 1 (Dissertativa)

Explique a diferença entre modo kernel e modo usuário em um sistema operacional. Como é realizada a transição entre eles? Cite dois exemplos de operações que exigem modo kernel.

Questão 2 (Dissertativa)

Descreva os cinco estados possíveis de um processo e explique as transições entre eles. Qual é a diferença entre um processo no estado 'ready' e no estado 'waiting'?

Questão 3 (Múltipla Escolha)

Sobre o escalonamento Round-Robin, qual das afirmativas é CORRETA? a) Round-Robin é não-preemptivo — o processo executa até terminar voluntariamente. b) Um quantum muito grande melhora o tempo de resposta para processos interativos. c) Cada processo recebe uma fatia de tempo (quantum) e, ao esgotá-la, vai para o final da fila READY. d) Round-Robin não garante fairness porque processos de alta prioridade sempre ficam em execução. e) O algoritmo do Banqueiro é uma variante do Round-Robin para múltiplos núcleos.

Questão 4 (Dissertativa)

Explique o que é uma condição de corrida (race condition) e como o uso de mutex a resolve. Por que simplesmente usar uma variável de flag booleana compartilhada não resolve o problema?

Questão 5 (Dissertativa)

Explique a diferença entre processo e thread quanto ao espaço de endereçamento, recursos compartilhados e custo de criação. Em que situação você escolheria threads em vez de processos para paralelizar uma tarefa?

Questão 6 (Dissertativa)

No problema do Jantar dos Filósofos com 5 filósofos, descreva exatamente como o deadlock pode ocorrer. Quais das quatro condições de Coffman estão presentes? Apresente uma solução que quebre o deadlock.

Questão 7 (Múltipla Escolha)

Quais das afirmativas sobre deadlock estão CORRETAS? I. Para ocorrer deadlock, as 4 condições de Coffman devem estar presentes simultaneamente. II. A Política do Avestruz consiste em detectar deadlocks e matar o processo mais antigo. III. O Algoritmo do Banqueiro é uma estratégia de evasão que requer que processos declarem seus recursos máximos antecipadamente. IV. Negar a condição de Espera Circular pode ser feito impondo uma ordenação total dos recursos e exigindo que processos os solicitem em ordem crescente. a) Apenas I e II b) Apenas I e III c) I, III e IV d) II, III e IV e) Todas

Questão 8 (Dissertativa)

Compare os modelos de thread N:1 e 1:1 quanto a: (a) paralelismo real em sistemas multicore, (b) comportamento quando uma thread realiza uma syscall bloqueante, e (c) custo de criação e troca de contexto. Qual modelo é mais comum nos SOs modernos?

Questão 9 (Múltipla Escolha)

Sobre virtualizacao, qual afirmativa e CORRETA? a) Um hipervisor Tipo 2 executa diretamente sobre o hardware sem SO hospedeiro. b) Containers Docker fornecem maior isolamento que VMs porque cada container tem seu proprio kernel. c) Um hipervisor Tipo 1 (bare-metal) tem menor overhead que o Tipo 2 porque acessa o hardware diretamente, sem intermediario. d) Paravirtualizacao exige que o SO convidado seja identico ao hospedeiro. e) VirtualBox e um exemplo de hipervisor Tipo 1.

Questão 10 (Dissertativa)

Liste as quatro estrategias para lidar com deadlocks e explique brevemente como cada uma funciona. Qual delas e adotada pelo Linux e Windows na pratica? Por que?

GABARITO COMENTADO

Questão 1 (Dissertativa):

O SO opera em dois modos de privilegio: modo kernel (supervisor), com acesso irrestrito ao hardware, e modo usuario, com acesso limitado para proteger o sistema. A transicao usuario->kernel ocorre por trap (instrucao especial que aciona uma interrupcao de software), chamada de sistema (syscall) ou interrupcao de hardware — o processador salva o contexto, muda o bit de modo e salta para o handler do kernel. Retorno: instrucao privilegiada de retorno de interrupcao (IRET no x86). Exemplos de operacoes que exigem kernel: acesso a disco (read/write de arquivo) e gerenciamento de memoria (mmap, malloc que chama brk).

Questão 2 (Dissertativa):

Os cinco estados sao: NEW (processo criado, aguardando admissao), READY (pronto para executar, aguardando CPU), RUNNING (executando na CPU), WAITING (bloqueado aguardando evento externo como I/O), TERMINATED (encerrado). Transicoes: NEW->READY (admitido); READY->RUNNING (dispatcher/escalonador escolhe); RUNNING->READY (preempcao por quantum ou prioridade); RUNNING->WAITING (solicita I/O ou evento); WAITING->READY (evento concluido, interrupcao); RUNNING->TERMINATED (exit()). Diferenca Ready vs. Waiting: processo READY tem todos os recursos menos a CPU — pode executar imediatamente se escalonado. Processo WAITING nao pode executar mesmo que receba a CPU — esta bloqueado aguardando evento externo (conclusao de I/O, sinal, etc.).

Questão 3 (Múltipla Escolha):

Alternativa CORRETA: c. Round-Robin e preemptivo — quando o quantum expira, o processo é forçado a liberar a CPU e volta para o final da fila READY. a) ERRADA: é preemptivo. b) ERRADA: quantum grande piora tempo de resposta (degenera em FCFS). d) ERRADA: RR puro não considera prioridade — todos recebem o mesmo quantum. e) ERRADA: Algoritmo do Banqueiro é estratégia de evasão de deadlock, não tem relação com RR.

Questão 4 (Dissertativa):

Race condition ocorre quando dois ou mais processos/threads acessam dados compartilhados concorrentemente e o resultado final depende da ordem de execução imposta pelo escalonador — não-determinístico e potencialmente incorreto. O mutex resolve porque lock e unlock são operações atômicas garantidas pelo hardware (instrução test-and-set ou similar): só uma thread consegue adquirir o mutex; as demais ficam bloqueadas (não desperdiçando CPU em busy waiting, dependendo da implementação). Uma variável flag booleana simples não funciona porque a própria leitura e escrita da flag não é atômica — duas threads podem ler flag=0 simultaneamente, ambas decidirem que podem entrar, e ambas entrarem na região crítica (o mesmo race condition que tentávamos evitar).

Questão 5 (Dissertativa):

Processo: espaço de endereçamento próprio e isolado, recursos privados (heap, pilha, descritores de arquivo), criação cara (fork copia tabela de páginas, mesmo com copy-on-write). Thread: espaço de endereçamento COMPARTILHADO dentro do processo, compartilha heap, variáveis globais e descritores — cada thread tem apenas pilha e registradores próprios, criação muito mais barata. Escolher thread quando: (1) as tarefas precisam compartilhar dados em alta velocidade (sem overhead de IPC); (2) a tarefa é paralelizável com dados compartilhados (ex: processar partes de um vetor); (3) o overhead de fork seria proibitivo (servidores web com muitas conexões). Escolher processo quando: isolamento de falhas é crítico (crash de uma instância não afeta as outras).

Questão 6 (Dissertativa):

Deadlock surge quando todos os 5 filósofos simultaneamente pegam o garfo à esquerda: cada um tem 1 garfo e espera pelo garfo direito (que está com o vizinho) — espera circular formada. Condições de Coffman presentes: (1) Exclusão Mútua: garfo só pode ser usado por um filósofo; (2) Segurar e Esperar: cada filósofo segura o garfo esquerdo e espera o direito; (3) Não Preempção: nenhum garfo pode ser tomado à força; (4) Espera Circular: F0 espera garfo de F1, F1 espera de F2, ..., F4 espera de F0. Solução por assimetria: filósofos de índice par pegam garfo esquerdo primeiro, ímpares pegam direito primeiro. Isso quebra a espera circular — pelo menos um filósofo consegue ambos os garfos e come, liberando-os depois.

Questão 7 (Múltipla Escolha):

Alternativa CORRETA: c (I, III e IV). I — CORRETA: as 4 condições de Coffman são necessárias simultaneamente para deadlock. II — ERRADA: Política do Avestruz IGNORA o problema (finge que não existe), não detecta nem mata processos. III — CORRETA: Banker's Algorithm requer declaração antecipada do máximo de recursos por processo. IV — CORRETA: ordenação total dos recursos quebra a espera circular eliminando ciclos no grafo de alocação.

Questão 8 (Dissertativa):

(a) Paralelismo: no modelo N:1, todas as N threads de usuário mapeiam para 1 thread de kernel — o SO vê apenas 1 thread e escalona ela em 1 núcleo; não há paralelismo real mesmo em multicore. No 1:1, cada thread de usuário tem uma thread de kernel, o SO pode escalonar threads diferentes em núcleos diferentes — paralelismo real. (b) Syscall bloqueante: no N:1, se uma thread faz read() bloqueante, TODA a thread de kernel bloqueia, parando TODAS as N threads de usuário do processo. No 1:1, apenas a thread que fez a syscall bloqueia; as demais continuam executando. (c) Custo: no N:1, criar/trocar threads é barato (feito em userspace pelo runtime). No 1:1, cada thread requer syscall ao kernel para criar e troca de contexto passa pelo kernel. Modelo moderno: 1:1 e o mais usado (Linux pthreads, Windows threads) pois o ganho de paralelismo justifica o custo adicional de criação em hardware multicore.

Questão 9 (Múltipla Escolha):

Alternativa CORRETA: c. Tipo 1 executa diretamente sobre o hardware, sem SO hospedeiro intermediando — logo, menor latência e maior desempenho. a) ERRADA: Tipo 2 executa SOBRE um SO hospedeiro (ex: VirtualBox roda no Windows). b) ERRADA: containers compartilham o kernel do host via namespaces — menor isolamento. VMs tem kernel próprio e maior isolamento. d) ERRADA: paravirtualização exige modificação do SO convidado para usar hypercalls, mas não exige que seja idêntico ao hospedeiro. e) ERRADA: VirtualBox e Tipo 2 (hosted). Tipo 1: VMware ESXi, Hyper-V, Xen.

Questão 10 (Dissertativa):

(1) Prevenção: projeta o sistema para garantir que ao menos uma das 4 condições de Coffman nunca ocorra. Ex: exigir que processos solicitem todos os recursos de uma vez (nega Segurar e Esperar). Restrição estrutural — recursos ficam subutilizados. (2) Evitação: antes de conceder recursos, analisa se o estado resultante é seguro. Algoritmo do Banqueiro e o exemplo clássico. Exige conhecimento prévio dos máximos. (3) Detecção e Recuperação: permite que deadlocks ocorram; periodicamente executa algoritmo de detecção de ciclo no grafo de recursos; ao detectar, abort ou rollback processos envolvidos. Caro em termos de overhead de recuperação. (4) Política do Avestruz: ignora o problema completamente — assume que deadlocks são raros demais para justificar o custo de tratamento. Linux e Windows adotam a Política do Avestruz na prática: deadlocks são suficientemente raros em aplicações comuns e o overhead de prevenção/detecção seria contínuo e custoso para todos os usuários, mesmo os que jamais encontrariam um deadlock.